SoSConfigurator

Interface Design Description: sosConfiguration HTTP/TLS/JSON

Abstract
Source-based description of the SoSConfigurator support system and its registered services. Documentation edition 5.3.0 retains the implementation and wire versions defined by the current primary source; no runtime upgrade or deployment is implied.

Contents

1 Documentation edition and source scope **3**

2 Service interface and operations **3**

3 Request and response data models **3**

4 Validation, errors and asynchronous behavior **4**

5 Console-only endpoint **4**

6 Security and access model **4**

7 Go and Rust implementation variants **4**

7.1 Go implementation 4

7.2 Rust implementation 4

8 Primary source references **5**

9 Revision history **5**

1 Documentation edition and source scope

This is documentation edition 5.3.0, dated 2026-11-09. No earlier `documentation/v5.2.2` set exists for this system. These documents were written from the primary Java source and secure configuration; no older system documentation was deleted or changed. Runtime/dependency/service versions remain source-defined: `SoSConfiguratorConstants.SYSTEM_VERSION` is 5.2.2 and service versions are 1.0.0. Documentation publication does not upgrade the runtime or deploy an SoS.

2 Service interface and operations

Service definition `sosConfiguration`, version 1.0.0, producer SoSConfigurator. The secure deployment advertises the HTTPS interface; the base path is `/api`. Payloads use JSON. `SosConfigurationAPI` sets `Cache-Control:no-store` on its JSON responses.

Method	Path	Success result
GET	`/api/catalog`	200, the catalog record.
GET	`/api/status`	200, relay and per-cloud executor status.
POST	`/api/plan`	200, the evaluated plan, including a refused plan.
POST	`/api/deployments`	202, object containing `plan` and workflow `id`.
GET	`/api/deployments`	200, up to 100 recorded submissions, newest first.
GET	`/api/deployments/{id}`	200, ColonyOS deployment/workflow view.

3 Request and response data models

SosRequest. `name` is a string; `clouds` is an array of member-cloud strings; `exchanges` is an array of Exchange records; `verifyDiscovery` is a boolean. Null clouds/exchanges are exposed as empty lists by model helpers and then evaluated by policy. Exchange has string fields `consumerCloud`, `consumer`, `providerCloud`, `provider`, `service`, and a string array `operations`. Policies determine whether the named members, partners and operation set are allowed.

Plan. Fields are `sos`, `members`, `allowed`, `violations`, `steps`, `notes` and `evaluation`. A Step contains `kind`, `role` and `target`. The evaluation retains catalog-derived cloud configuration. A refused plan has no executable step list. A submission response includes the plan and the returned workflow identifier; it does not assert workflow completion.

Catalog. Top-level fields are `organization`, `colony`, `relay`, `forbiddenConsumers` and `clouds`. Cloud records include organization/implementation, executor role, inter-cloud capability, service ports, approved consumers and exports. The catalog is the policy authority; it is not an arbitrary request-provided configuration.

Status and submissions. Status returns `relay` and a `clouds` map. An executor I/O failure becomes an object with `state=offline` and `error` rather than failing every executor's observation. Submission-list rows contain `id`, `sos`, `submittedBy` and `submittedAt`. Workflow/process state is returned by the ColonyOS adapter; it is not replaced by the local submission record.

4 Validation, errors and asynchronous behavior

Missing, empty or invalid JSON request bodies give 400 with an `error` string. Bodies exceeding 65536 bytes give 413. A plan that violates policies is returned with 200 by the plan operation; submitting that plan gives 422 with `error` and `plan`. Submission/backend I/O errors give 502. A deployment identifier not matching 64 lowercase hexadecimal characters gives 404. Authentication/authorization filters can reject requests earlier with 401/403. Unknown routes and console-only paths on the Arrowhead connector are not service operations.

All response objects are serialized with the system mapper; omitted null fields and source record types remain significant. A timeout or local database error after remote workflow creation can require operator reconciliation. No idempotency or exactly-once guarantee is introduced by these interfaces.

5 Console-only endpoint

`POST/api/session` performs browser console login and manages the `ah_sos_console` session cookie under console-specific controls. It is not registered as a `sosConfiguration` service operation and is not an Arrowhead identity login. UI/session paths exist only on the console connector; consumers of the registered service must use the Arrowhead identity/security model.

6 Security and access model

The secure Arrowhead connector uses TLS with client certificates, outsourced identity verification and token-to-certificate binding. `SosAccessFilter` permits a `sosConfiguration` operation for a verified Sysop, a named `sos.api.whitelist` system or a ConsumerAuthorization operation grant. An authenticated identity alone is insufficient. General management remains governed by the common management filter.

The separate console authenticates a browser with a configured scrypt password hash and the `ah_sos_console` cookie. It applies Host/Origin checks and session/lockout rules. A console session does not become an Arrowhead identity token. Static UI/session paths are console-only; the Arrowhead connector returns 404 for them. Console handling marks authenticated operator requests so the internal API can execute without pretending to be an mTLS system caller.

7 Go and Rust implementation variants

7.1 Go implementation

No `Eclipse-Arrowhead-go/ah5-SoSConfigurator-go` folder exists in the designated primary Go tree. Go local-cloud systems may be targets of a Java SoSConfigurator only if their catalog and inter-cloud implementation meet the policy and executor checks. This is not a Go implementation of SoSConfigurator.

7.2 Rust implementation

No `Eclipse-Arrowhead-rust/ah5-SoSConfigurator-rust` folder exists in the designated primary Rust tree. The policy rejects a catalogued implementation without the required Gatekeeper/GateTunnel inter-cloud capability; the mere presence of Rust local-cloud roles does not establish eligibility or implement this system.

8 Primary source references

All source paths below are relative to `Eclipse-Arrowhead-java/ah5-SoSConfigurator-java`.

- `src/main/java/eu/arrowhead/sosconfigurator/SoSConfiguratorSystemInfo.java` and `SoSConfiguratorConstants.java`: registration, names, versions and paths.
- `api/http/SosConfigurationAPI.java`, `api/http/GeneralManagementAPI.java`, `policy/SosRequest.java`, `catalog/Catalog.java`, `planner/Planner.java` and `service/SosConfigurationService.java`, under the same Java package: operations, DTOs, planning and persistence.
- `security/SosAccessFilter.java` and `console/ConsoleFilter.java` under that package: caller and connector boundaries.
- `config/application.properties`, packaged `src/main/resources/application.properties` and `SoSConfiguratorDefaults.java`: secure overrides versus development/source defaults.
- `vendor/common-utils` and `vendor/data-transfer-objects`: common management, security and native sysMonitor publisher contract.

9 Revision history

2026-11-09: Initial source-based LaTeX documentation edition 5.3.0. Review draft; document compilation and source traceability checked.